Proyecto Parte I: Fórmula E CES Tec

Institución: Tecnologico de Costa Rica

Curso: Fundamentos de Sistemas Computacionales, Semestre II 2026

Profesor: Milton Villegas Lemus Asistente: Asly Barahona Maroto

Integrantes: Jose Matías Marín Vega 2026087935 / Josué David González Urbina 2026081800

1. Introducción

La telemetría es la tecnología que permite medir magnitudes físicas de forma remota y enviar esa información, o recibir órdenes, desde un operador. Nació en la Fórmula 1 y hoy se utiliza también en la Fórmula E, la categoría de monoplazas eléctricos que busca impulsar la movilidad sostenible. Inspirado en ese concepto, este proyecto aplica la idea de control remoto a un carro de juguete.

El objetivo general es construir un proyecto que integre hardware y software, aplicando conceptos básicos de la Ingeniería en Computadores. Para ello se adaptó un carro a baterías con dos motores DC: uno de tracción (adelante y atrás) y otro de dirección (izquierda y derecha). El carro incorpora seis LED (dos luces delanteras, dos traseras y dos direccionales) y un sensor de luz exterior basado en una fotorresistencia (LDR).

El cerebro del sistema es una Raspberry Pi Pico W programada en MicroPython. Esta recibe comandos escritos en la consola de Thonny y genera las señales eléctricas para los demás componentes: un driver de motores L298N, un registro de corrimiento 74LS164 que controla los LED con solo dos pines, y la entrada analógica del sensor de luz. La velocidad se controla mediante modulación por ancho de pulso (PWM). Se implementaron los comandos de mando (pwm, dir, lf, lb, ll, lr, ldr) y los patrones de movimiento (circle, infinite, zigzag y especial).

Esta primera parte del proyecto se centra en el hardware y en el firmware local del carro. Más adelante se espera incorporar la comunicación inalámbrica con una aplicación en el computador, siguiendo la arquitectura de la especificación (socket cliente en Python, lógica de juego e interfaz de usuario).

Este documento presenta el análisis de resultados (diagramas de módulos, plan de pruebas, reglas de grupo, roles y fechas), las conclusiones, las recomendaciones, las fuentes consultadas y la bitácora de trabajo.

2. Análisis de Resultados

2.1 Diagramas de módulos

2.1.1 Diagrama de arquitectura de hardware

El sistema se divide en cuatro subsistemas: alimentación, control, potencia/motores e iluminación

Módulo	Componente	Función	Conexión con la Pico W
Alimentación lógica	Batería 9V + regulador L7805CV + interruptor	Entrega 5 V estables a la Pico, al 74LS164 y a la lógica del L298N	VSYS / VBUS (5 V) y GND común
Alimentación de potencia	3 pilas AA en serie (4.5 V) + interruptor	Alimenta los dos motores a través del L298N	Solo al L298N; GND común con el resto
Control	Raspberry Pi Pico W (MicroPython)	Interpreta comandos, genera PWM y controla las luces	Pines GP14 a GP21 y GP26
Driver de motores	L298N (puente H doble)	Invierte polaridad y regula potencia de los motores	GP16 (ENA), GP17 (IN1), GP18 (IN2), GP19 (ENB), GP20 (IN3), GP21 (IN4)
Motor 1	Motor DC	Tracción (adelante / atrás)	Salidas OUT1 y OUT2 del L298N
Motor 2	Motor DC	Dirección (derecha / izquierda)	Salidas OUT3 y OUT4 del L298N
Iluminación	74LS164 + 6 LED + 6 resistencias de 330 Ω	Controla las luces con solo 2 pines	GP15 (A y B unidas, datos) y GP14 (CLK); CLR a 5 V
Sensor de luz	LDR + resistencia de 10 kΩ (divisor de tensión)	Detecta baja luminosidad	GP26 (ADC0), LDR a 3,3 V y 10 kΩ a GND

2.1.2 Asignación de pines

Pin Pico W	Señal	Destino
GP14	CLK	74LS164, pin 8
GP15	DATA	74LS164, pines 1 y 2 (A y B)
GP16	ENA (PWM, 1 kHz)	L298N, habilita el Motor 1
GP17	IN1	L298N, sentido del Motor 1
GP18	IN2	L298N, sentido del Motor 1
GP19	ENB (PWM, 1 kHz)	L298N, habilita el Motor 2
GP20	IN3	L298N, sentido del Motor 2
GP21	IN4	L298N, sentido del Motor 2
GP26 (ADC0)	Lectura LDR	Divisor de tensión LDR / 10 kΩ

2.1.3 Mapa de salidas del registro de corrimiento 74LS164

Salida	Bit	LED	Función en el código
QA, QB	0 y 1	LED1, LED2	Luces traseras / de freno (LUZ_FRENO)
QC, QD	2 y 3	LED3, LED4	Luces delanteras / de retroceso (LUZ_RETRO)
QE	4	LED5	Direccional izquierda (DIR_IZQ)
QF	5	LED6	Direccional derecha (DIR_DER)
QG, QH	6 y 7	No usadas	Siempre en 0

2.1.4 Diagrama de módulos de software

Módulo (función)	Responsabilidad	Depende de
Configuración de pines y constantes	Define los pines PWM, los pines digitales, el ADC, los niveles de potencia (NIVELES), los tiempos y el umbral del LDR	machine, utime
actualizar_luces()	Arma el byte de luces según el estado (manual, LDR, freno, retroceso, direccionales) y lo envía al 74LS164	Variables de estado, DATA, CLK
tick() (Timer 500 ms)	Alterna el parpadeo de las direccionales, lee el LDR y refresca las luces	actualizar_luces(), ADC
set_pwm(valor)	Controla la tracción de −4 a 4 con PWM, y activa el freno temporizado al detenerse	ENA, IN1, IN2
aplicar_direccion(), mover_direccion(), set_dir()	Controla el motor de dirección por tiempo (sin sensor de posición), lo mantiene energizado con menor potencia cuando está girado, y soporta el valor 2 para retroceder	ENB, IN3, IN4
Patrones: circle(), infinite(), zigzag(), especial()	Secuencias de movimiento combinando dirección, tracción y direccionales	set_pwm(), set_dir(), direccionales()
ejecutar(linea)	Interpreta y valida los comandos de la consola	Todos los anteriores
apagar_todo(), iniciar()	Detiene motores, centra la dirección, apaga luces y limpia el registro	Todos los anteriores

Módulo (función)	Responsabilidad	Depende de
Programa principal	Inicia el Timer y ejecuta el ciclo de lectura de comandos con input()	ejecutar()

2.1.5 Decisiones de diseño relevantes

- PWM de tracción: los niveles 1 a 4 se asignan a duty de 32000, 43000, 54000 y 65535 (de 65535). El nivel más bajo es el mínimo con el que los motores del carro logran arrancar.
- Dirección por tiempo: como no hay sensor de posición, el programa recuerda dir_actual y gira el motor T_DIR (0,4 s) por cada paso necesario. En las posiciones izquierda y derecha mantiene el motor energizado con PWM reducido (55000) para que las llantas no regresen al centro.
- Luces de freno: al pasar de movimiento a detención, las luces traseras permanecen encendidas T_FRENO (1000 ms).
- Luces de retroceso: al retroceder se encienden los LED frontales/retroceso.
- Sensor de luz: con ldr 1, si la lectura es menor a UMBRAL_LDR (20000) se considera oscuro y se encienden las luces. Con ldr 0 el modo se desactiva.
- Timer sin colisiones: la bandera ocupado evita que la interrupción del Timer interrumpa el envío de bits al 74LS164.

2.2 Plan de pruebas

Las pruebas se realizaron primero por módulos en protoboard y luego con el sistema integrado en el carro.

N.º	Módulo	Prueba	Procedimiento	Resultado esperado	Resultado obtenido
P1	Alimentación	Voltaje del regulador	Medir con multímetro la salida del L7805CV con la batería de 9 V	5 V ± 0,25 V	5V
P2	Alimentación	Voltaje de pilas	Medir las 3 pilas AA en serie	≥ 4.5 V	4,5V
P3	Alimentación	Interruptores	Accionar cada interruptor	Corta y restablece cada alimentación	Funcionamiento correcto

N.º	Módulo	Prueba	Procedimiento	Resultado esperado	Resultado obtenido
P4	Pico W	Comunicación con consola	Conectar la Pico a Thonny y ejecutar el programa	Aparece el prompt > y los mensajes de ayuda	Funcionamiento correcto
P5	Luces	Limpieza del registro	Ejecutar stop	Todos los LED apagados	Funcionamiento correcto
P6	Luces	Luces frontales	lf 1 y lf 0	Se encienden y apagan LED3 y LED4	Funcionamiento correcto
P7	Luces	Luces traseras	lb 1 y lb 0	Se encienden y apagan LED1 y LED2	Funcionamiento correcto
P8	Luces	Direccional izquierda	ll 1 y ll 0	LED5 parpadea cada 500 ms	Funcionamiento correcto
P9	Luces	Direccional derecha	lr 1 y lr 0	LED6 parpadea cada 500 ms	Funcionamiento correcto
P10	Luces	Luces de freno	pwm 2 y luego pwm 0	Luces traseras encendidas ~1 s tras detenerse	Funcionamiento correcto
P11	Luces	Luces de retroceso	pwm -2	Se encienden las luces de retroceso mientras retrocede	Funcionamiento correcto
P12	Motor de tracción	Adelante	pwm 1 a pwm 4	Avanza; la velocidad aumenta con cada nivel	Funcionamiento correcto
P13	Motor de tracción	Atrás	pwm -1 a pwm -4	Retrocede; la velocidad aumenta con cada nivel	Funcionamiento correcto
P14	Motor de tracción	Detención	pwm 0	El motor se detiene	Funcionamiento correcto
P15	Motor de dirección	Izquierda y derecha	dir -1, dir 1, dir 0	Las llantas giran, se sostienen y regresan al centro	Funcionamiento correcto

N.º	Módulo	Prueba	Procedimiento	Resultado esperado	Resultado obtenido
P16	Motor de dirección	Retroceso por dirección	dir 2	El carro retrocede (equivale a pwm -2)	Funcionamiento correcto
P17	Sensor de luz	Control automático	ldr 1, tapar y destapar el sensor	Con poca luz se encienden 4 LED; con luz se apagan	Funcionamiento correcto
P18	Sensor de luz	Desactivación	ldr 0 con el sensor tapado	Las luces no se encienden por el LDR	Funcionamiento correcto
P19	Patrones	Circle	circle	Vuelta circular y detención en el punto de partida	Funcionamiento correcto (Por motivo del largo del cable no se probó en suelo)
P20	Patrones	Infinite	infinite	Recorrido en forma de 8 invertido	Funcionamiento correcto (Por motivo del largo del cable no se probó en suelo)
P21	Patrones	ZigZag	zigzag	Se aprecia el movimiento alternado	Funcionamiento correcto (Por motivo del largo del cable no se probó en suelo)
P22	Patrones	Especial	especial	Avanza, hace una S, y retrocede con luces de emergencia	Funcionamiento correcto (Por motivo del largo del cable no se probó en suelo)
P23	Validación	Comandos inválidos	pwm 9, dir 5, xyz, pwm	Valor limitado o mensaje de error, sin fallar el programa	Funcionamiento correcto
P24	Ensamblado	Carcasa desmontable	Conectar y desconectar la carcasa de la base	Se desmonta y se conecta sin dañar las conexiones	Funcionamiento correcto
P25	Integración	Prueba completa	Ejecutar todos los comandos con el sistema en el carro	Todo funciona con alimentación propia	Funcionamiento correcto

2.3 Reglas de grupo

1. Las reuniones de trabajo se realizan al menos 1 veces por semana, de forma presencial
2. Ningún integrante sube cambios que dejen el programa sin funcionar. Se prueba antes.
3. El código se documenta con comentarios claros para que ambos integrantes puedan explicarlo en la defensa.
4. Las tareas se reparten por acuerdo.

2.4 Roles

Integrante	Rol principal	Responsabilidades
Josue David	Hardware y ensamblado	Circuito en protoboard, soldaduras, montaje de LED y LDR en la carcasa, alimentación e interruptores
Jose Matías	Software	Programación en MicroPython, comandos, patrones, uso de GitHub
Ambos	Pruebas y documentación	Plan de pruebas, calibración de constantes, bitácora y documento final.

3. Conclusiones

1. Se logró integrar hardware y software en un carro de juguete controlado por una Raspberry Pi Pico W. El carro ejecuta los comandos de la Tabla 1 de la especificación y los cuatro patrones de movimiento.
2. El registro de corrimiento 74LS164 permitió controlar seis LED con solo dos pines de la Pico. Esto deja pines libres y demuestra cómo una estructura digital simple amplía las salidas de un microcontrolador.
3. El PWM resultó eficaz para simular diferentes velocidades. Sin embargo, los motores de juguete solo arrancan por encima de cierto duty, por lo que los niveles de potencia se calibraron experimentalmente (de 32000 a 65535) en lugar de repartirse desde cero.
4. El divisor de tensión con la fotorresistencia permitió automatizar las luces. El umbral (UMBRAL_LDR) depende de las condiciones de iluminación del lugar y debe ajustarse.
5. Separar la alimentación en una fuente de 4,5 V para los motores y otra de 5 V regulada para la lógica reduce el riesgo de reinicios de la Pico por caídas de tensión cuando arrancan los motores. Es indispensable que todas las tierras sean comunes.
6. Usar un Timer de 500 ms para el parpadeo y la lectura del LDR permitió que las luces funcionen de forma independiente de los movimientos del carro.
7. Probar por módulos antes del ensamblado final facilitó detectar fallas de conexión y de lógica. También fue útil para este proyecto trabajar con una estructura clara de funciones.

4. Recomendaciones

1. Probar por módulos. Armar y verificar cada parte en protoboard (luces, motores, LDR) antes de integrarlas.
2. Verificar la tierra común. La mayoría de las fallas extrañas (motores que no responden, LED erráticos) se deben a tierras no compartidas entre la Pico, el L298N y las baterías.
3. Cuidar la alimentación. Medir las baterías antes de cada prueba y llevar pilas de repuesto. Con pilas bajas los motores pierden fuerza y los tiempos de giro dejan de ser reproducibles. Revisar que el regulador L7805CV no se caliente en exceso.
4. Calibrar las constantes en el lugar de uso. NIVELES, T_DIR, T_CIRCULO, T_ZIGZAG y UMBRAL_LDR dependen del carro, del piso y de la luz.
5. Asegurar el cableado. Soldar los LED y el LDR a cables flexibles, rotular los cables y fijar las baterías para que no se muevan con el carro.
6. Usar bases para circuitos integrados (74LS164) para poder reemplazarlos sin desoldar. Se recomienda trasladar el circuito a una placa preperforada si se dispone de tiempo.
7. Proteger los motores. Evitar mantener el motor de dirección empujando a plena potencia contra el tope mecánico. Si se calienta, reducir DUTY_MANTENER.

5. Fuentes Consultadas

Raspberry Pi Ltd. (s. f.). *Raspberry Pi Pico W datasheet*. https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf

MicroPython. (s. f.). *Quick reference for the RP2*. https://docs.micropython.org/en/latest/rp2/quickref.html